1. Title Page

Course Name: Object-Oriented Programming using Java
Project Title: Console-Based Library Management System
Student Name: Trishit Ghosh
Roll Number: 66 **Enrollment Number:** 12024052017068
Instructor Name: Dr. Soumadip Biswas
Semester: Spring 2026

2. Problem Statement

The purpose of this project is to design and implement a **Library Management System (LMS)** using Java, emphasizing Object-Oriented Programming (OOP) principles. The system is intended to automate essential library operations such as book catalog management, user registration, book issuing and returning, and fine calculation.

Traditional library systems often rely on manual record-keeping or outdated software, which leads to inefficiencies such as data inconsistency, lack of scalability, and difficulty in tracking transactions. This project addresses these challenges by providing a structured, modular, and maintainable software solution.

The system uses **CSV-based file storage** instead of a database to ensure simplicity and minimal setup cost, making it suitable for small-scale environments such as schools or personal libraries.

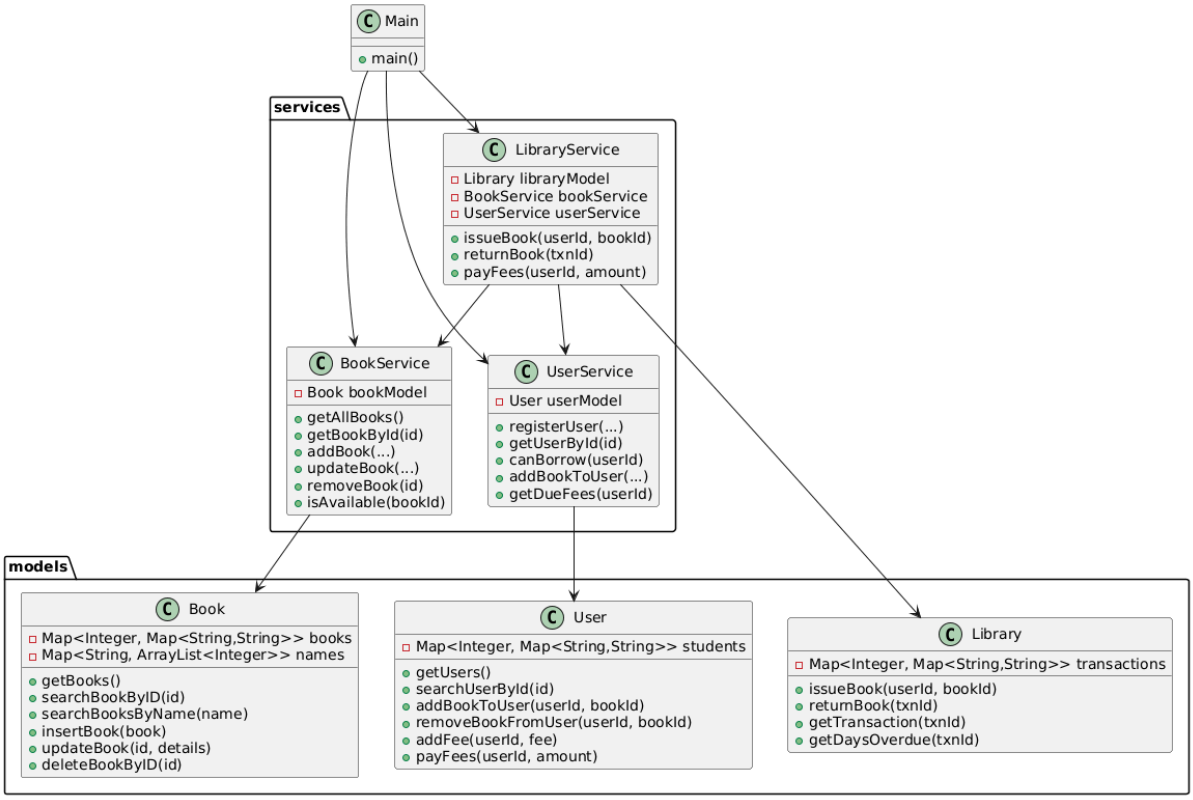
Key objectives of the system include:

- Efficient management of books and users
- Tracking of issued and returned books
- Enforcement of borrowing limits
- Automatic fine calculation for overdue books
- Modular architecture for easy scalability

The system ensures data persistence, logical separation of components, and robustness through validation and error handling.

3. System Design

3.1 UML Class Diagram



3.2 Description of Major Classes

Book Class:
Responsible for managing book records. It handles CSV reading/writing, searching by various attributes, and updating book data.

User Class:
Maintains user information including borrowed books and due fees. It supports registration, updates, and fee management.

Library Class:
Handles transaction records such as issuing and returning books. It also calculates overdue days and maintains transaction status.

Service Classes:

- **BookService:** Acts as an interface between UI and Book model
- **UserService:** Handles user-related logic such as borrowing limits
- **LibraryService:** Coordinates between book and user services to enforce business rules

3.3 Package Structure

```
models/
 Book.java
 User.java
 Library.java

services/
 BookService.java
 UserService.java
  LibraryService.java

Main.java
```

3.4 Interaction Flow

1. User selects an option from the console menu
2. The Main class calls the appropriate service
3. The service validates business rules
4. Model classes interact with CSV files
5. Response is returned to the user

Example Flow:

```
Main → LibraryService.issueBook()
       → UserService.canBorrow()
       → BookService.isAvailable()
       → Library.issueBook()
```

4. OOP Concepts Demonstrated

4.1 Encapsulation

Where Used: Model classes (Book, User, Library)
Why Used: To protect internal data structures and ensure controlled access

```
private Map<Integer, Map<String, String>> books = new HashMap<>();

public Map<Integer, Map<String, String>> getBooks() {
   return books;
}
```

Explanation:
The data is hidden within the class and accessed via public methods, ensuring data integrity and preventing unauthorized modifications.

4.2 Abstraction

Where Used: Service layer
Why Used: To hide implementation details from the user

```
public boolean addBook(String name, String genre, String author, String year, int copies) {
    int newId = bookModel.getNextId();
    return bookModel.insertBook(...);
}
```

Explanation:
The user interacts with simple methods without knowing how data is stored or processed internally.

4.3 Polymorphism

Where Used: Method behavior based on inputs
Why Used: To allow flexibility in operations

```
public boolean updateUser(int id, String name, String className)
```

Explanation:
Different inputs produce different outcomes while using the same method structure.

5. Implementation Highlights

5.1 Book Availability Check

```
if (!bookService.isAvailable(bookId)) {
    return "No copies available for this book.";
}
```

Justification:
Prevents issuing books when no copies are available.

5.2 Fine Calculation

```
long overdueDays = libraryModel.getDaysOverdue(txnId);
double fine = overdueDays * 5.0;
```

Justification:
Implements dynamic fine calculation based on delay.

5.3 Borrow Limit Enforcement

```
return borrowed.size() < MAX_BOOKS;
```

Justification:
Ensures fair usage by limiting number of borrowed books.

5.4 CSV Data Persistence

```
FileWriter csv = new FileWriter(FILE_PATH, true);
```

Justification:
Provides lightweight storage without requiring a database.

6. Testing & Error Handling

6.1 Test Cases

Test Case	Expected Result
Add Book	Book successfully added
Issue Book	Issued if available
Return Book	Fine applied if overdue
Invalid User ID	Error message

6.2 Edge Cases

- Borrow limit exceeded
- Duplicate book issue
- Invalid input values
- Empty CSV file
- Book not found

6.3 Failure Scenarios

- File read/write failure
- Incorrect numeric input
- Missing transaction records

```
catch (NumberFormatException e) {
   return 0;
}
```

7. Git Workflow

Commit History (Expected)

1. Initial CSV setup
2. Model implementation
3. Service layer development
4. Main menu integration
5. Testing and debugging

Development Progression

- Step 1: Designed data structure
- Step 2: Implemented models
- Step 3: Added service layer
- Step 4: Integrated UI
- Step 5: Performed testing and fixes

8. Conclusion & Future Scope

Conclusion

The project successfully demonstrates the application of Object-Oriented Programming concepts in building a modular and scalable system. The use of service and model layers ensures separation of concerns, while CSV-based persistence provides a lightweight storage solution.

The system is efficient, maintainable, and suitable for small-scale library environments.

Future Scope

- Integration with relational databases (MySQL/PostgreSQL)
- Development of GUI using JavaFX
- Implementation of REST APIs
- User authentication system
- Advanced reporting and analytics
- Email/SMS notification system